

# Spike Extension Report

## Mohamed Shafi Uzman Fassy (102608927)

### COS30031 – Games Programming

#### Introduction

The purpose of this spike extension report is to communicate the extra spike tasks that I have undertaken on my own on this unit and discuss how those tasks align with the Intend Learning Outcomes and What I have learned and achieved from those extension tasks.

#### Intended Learning Outcomes

1. **Design (D)**: Discusses game engine components including architectures of components, selection of components for a particular game specification, the role and purpose of specific game engine components, and the relationship of components with underlying technologies.
2. **Implementation (I)**: Creates games that utilize and demonstrate game engine component functionality, including the implementation of components that encapsulate specific low-level APIs.
3. **Performance (P)**: Identifies performance bottlenecks by using profiling techniques and tools and applying optimization strategies to improve performance.
4. **Maintenance (M)**: Explains and illustrates the role of data structures and patterns in game programming and rationalizes the selection of these for the development of a specified game scenario.

#### Spike Extension Alignment

| Spike Extension                  | ILO 1 | ILO 2 | ILO 3 | ILO4 |
|----------------------------------|-------|-------|-------|------|
| Task 4 Grid World Multi-Threaded |       | X     | X     |      |

## Justification For Each Extension

### Task 4: Grid World Multi-Threaded.

```
int main()
{
    displayWelcomeMessage();

    // Initialize grid
    char grid[Grid_Size][Grid_Size] = {
        {'#', '#', '#', '#', '#', '#', '#', '#'},
        {'#', 'G', ' ', ' ', 'D', ' ', ' ', '#'},
        {'#', ' ', ' ', ' ', ' ', ' ', ' ', '#'},
        {'#', '#', '#', ' ', ' ', ' ', '#', '#'},
        {'#', ' ', ' ', ' ', ' ', ' ', ' ', '#'},
        {'#', ' ', '#', '#', '#', ' ', ' ', '#'},
        {'#', ' ', ' ', ' ', ' ', ' ', ' ', '#'},
        {'#', '#', 'S', '#', '#', '#', '#', '#'};

    displayGrid(grid, playerPosition);

    // Starts threads
    thread inputThread(inputThreadFunction);
    thread gameThread(gameLoopThreadFunction, grid);

    // Waits for threads to finish
    inputThread.join();
    gameThread.join();

    // Game over messages displays
    if (isPlayerWon)
    {
        cout << "Thanks for playing. You Won!." << endl;
    }
    else
    {
        cout << "Thanks for playing. There probably wont be a next time." << endl;
    }

    return 0;
}
```

I implemented multi-threading in the Grid World program that was created in task 3 by implementing threading and improve performance. Task 4 incorporates 2 learning outcomes **Implementation** and **Performance** where we use implement multi-threading to showcase the ability to create a responsive non-blocking game loop with the use of threads, mutexes, atomic variables and condition variables. For Performance we used **atomic Boolean flags** such as **inputReceived** to synchronize game loops with user input to reduce unnecessary processing if no input has been received which optimizes CPU usage as the thread sleeps when no user input is found and therefore does not consume resources.

## Conclusion

In conclusion, the spike extension reports should provide a comprehensive overview of the key learning outcomes used to achieve task 4 spike extension and align closely with the ILOs of this unit. Unfortunately, I could not achieve more spike extension due to poor time management, but I hope this extension report highlights some of my skills that I have learned and put towards the extension task I demonstrated.